Dizajn sistema
eSportskiCentar

Milan Aleksić
Saša Savić
Aldin Tursić
Igor Grubiša
Milan Ignjatić

Sadržaj

1. Uvod ... 3
1.1. Namjena sistema ... 3
1.2. Projektni ciljevi ... 4
1.3. Definicije, skraćenice ... 4
1.4. Referentni dokumenti ... 5
1.5. Kratak pregled dokumenta i predložene arhitekture ... 5
2. Arhitektura postojećeg sistema ... 6
2.1. Dvoslojna arhitektura ... 6
2.2. Troslojna arhitektura ... 6
3. Predložena arhitektura ... 8
3.1. Kratak pregled arhitekture I funkcionalnosti podsistema ... 8
3.2. Dekompozicija sistema ... 10
3.3. HW/SW mapiranje ... 12
3.3.1. Dijagram komponenata ... 12
3.3.2. Dijagram razmještaja ... 14
3.4. Perzistentni sloj ... 15
3.5. Kontrola prava pristupa i sigurnost ... 21
3.5.1. Podsistem za administratora ... 21
3.5.2. Podsistem za upravnika ... 24
3.5.3. Podsistem za dežurnog radnika ... 27
3.5.4. Podsistem za klijenta ... 30
3.6. Konkurentnost i kontrola toka ... 32
3.7. Granična stanja sistema ... 32

1. Uvod

1.1. Namjena sistema

Sistem eSportskiCentar je zamišljen kao web aplikacija koja je prvenstveno namjenjena radnicima i upravi sportskih centara. Aplikacija vrši raspoređivanje sportskih događaja po raspoloživim terminima ili terminima unesenim od strane korisnika. Aplikacija podržava rad četiri tipa korisnika: administrator sistema, upravnik sportskog centra, dežurni radnik i klijent. Kao što je prethodno navedeno, aplikacija je prvenstveno namjenjena organizaciji samog sportskog centra pa je skup slučajeva upotrebe za spoljašnje korisnike sveden samo na operaciju pregleda rasporeda događaja i slanje zahtjeva za zakazivanje događaja. Ova funkcionalnost je od izrazitog značaja za publiku sportskih događaja, kao i za sportska društva. Administrator ima širok spektar mogućnosti i ovaj tip korisnika je zadužen za održavanje svih hiperpodataka o sportskom centru. Ima mogućnost pregleda i unosa novih dvorana (terena, organizacionih jedinica, igrališta) unošenjem osnovnih parametara o dvorani (uključuje dimenzije, opremu, tip podloge, kapacitet i sve ostale značajne parametre). Sistem na osnovu podataka automatski određuje za koje tipove događaja je dvorana odgovarajuća. Ovaj jednostavan mehanizam na značajan način sprječava greške u daljem radu i funkcionisanju sportskog centra. Administrator takođe ima ulogu u svim poslovima vezanim za korisničke naloge. Pošto radnicima i upravnicima nije potrebna registracija, administrator vrši registraciju svih navedenih tipova naloga i vrši operacije nad tim nalozima (osnovne operacije koje danas predstavljaju minimalan standard: pregled, brisanje, ažuriranje,...). Sa druge strane, klijentima je dovoljno da običnom registracijom kreiraju svoj nalog.

Sada dolazimo do najbitnijeg tipa naloga kome ovaj sistem u značajnoj mjeri olakšava posao. Upravnik je glavno odgovorno lice za posebnu organizacionu jedinicu (dvoranu). Ima mogućnost dodavanja novih događaja u sistem te njihov pregled. Događaji uglavnom predstavljaju pojedinačnu utakmicu koja se sastoji od termina održavanja. Mogućnost registracije takmičenja ima ulogu u lakšem unosu podataka za događaj (imena organizatora, domaće i gostujuće ekipe već postoje u sistemu pa se mogu izabrati iz padajućeg menija) i lakšem pregledu, odnosno skupu podataka za izvještaj.

Jako bitna interakcija je između upravnika i radnika u kojoj upravnik može slati zadatke u vidu notifikacije radnicima, kao i rok do kog je neophodno izvršiti zadatak. Zadaci vezani za događaje se automatski šalju radniku jer su predefinisani za svaki tip događaja.

1.2. Projektni ciljevi

Glavni cilj je kreiranje web aplikacije koja će biti intuitivna za korišćenje i biti jednostavna za obuku korisnika. Organizacija i sam rad sportskih centara će biti značajno olakšan i modernizovan. Fokus je stavljen na sljedeće projektne ciljeve:

- Sigurnost – Web aplikacija će za komunikaciju klijenta i servera koristiti HTTP protokol koji je danas među najzastupljenijim i najsigurnijim protokolima aplikativnog sloja TCP/IP modela. Eksterni korisnik neće imati pristup kritičnim funkcionalnostima, a sve naloge koji će imati pristup kritičnim informacijama registruje isključivo administrator sistema.
- Performanse – sistem mora u razumnom vremenskom roku opsluživati zahtjeve klijenata. Performanse mogu zavisiti i od internet konekcije, kao i od hardverskih specifikacija uređaja.
- Za korištenje je potrebno imati bilo koji uređaj sa instaliranim Windows ili Linux operativnim sistemom. Potrebno je takođe imati instaliran i web browser preko kojeg se pristupa web aplikaciji. Da bi sistem bio dostupan, potrebna je još i internet konekcija.
- Dostupnost sistema – sistem treba da bude dostupan sve vrijeme.
- Pouzdanost sistema – sistem treba da bude otporan na greške i da se svaka greška može brzo otkloniti.
- Lako održavanje – sistem mora biti pogodan za održavanje i za dodavanje novih funkcionalnosti inkrementalno.

1.3. Definicije, skraćenice

U ovom poglavlju ćemo definisati manje poznate i domenske pojmove, kao i skraćenice.

Sistem - organizovani skup elemenata ili komponenti koje zajedno rade kako bi obavljale određene funkcije ili postigle određene ciljeve

Server - računarski sistem koji obavlja određene zadatke ili pruža usluge drugim računarskim entitetima kao što su klijenti ili druge aplikacije

Klijent - Klijent je entitet koji koristi usluge ili resurse koje pruža server u nekom sistemu

Aplikacija - Aplikacija je softverski program dizajniran da obavlja određene zadatke ili pruža određene funkcionalnosti korisnicima

Web aplikacija - vrsta aplikacije koja se izvršava na web pregledačima i nije instalirana na lokalnoj mašini već na udaljenom serveru

Browser - (srp. Web pregledač) - softverska aplikacija koja omogućava korisnicima da pristupaju web stranicama i sadržaju na internetu (primjeri su Firefox, Chrome, Microsoft Edge...)

Arhitektura sistema - odnosi se na organizaciju i strukturu elemenata i komponenti unutar jednog sistema. To obuhvata dizajniranje, planiranje i organizovanje kako bi se postigla određena funkcionalnost, performanse, skalabilnost, pouzdanost i druge karakteristike sistema.

Baza podataka - organizovana kolekcija podataka koja se koristi za njihovo efikasno skladištenje, upravljanje i pristupanje.

HTTP - Hypertext Transfer Protocol

TCP/IP - Transmission Control Protocol/Internet Protocol (Skup protokola koji omogućava komunikaciju i razmjenu podataka između računara na internetu).

SRS - Software Requirements Specification

1.4. Referentni dokumenti

Specifikacija softverskih zahtjeva (SRS) - eSportskiCentar

1.5. Kratak pregled dokumenta i predložene arhitekture

Detaljniji opis sistema koji je predstavljen u uvodnom dijelu je dat u sklopu SRS dokumenta dok su ovdje navedeni osnovni ciljevi, osnovne funkcionalnosti u vidu neformalne specifikacije, manje poznate riječi i skraćenice i referentni dokumenti. U drugom poglavlju je opisana arhitektura postojećeg sistema. Treće poglavlje bilježi kratak pregled arhitekture i funkcionalnosti sistema, dekompoziciju, hardversko i softversko mapiranje zajedno sa dijagramima komponenata i razmještaja, perzistentni sloj, prava pristupa i sigurnosti, kontrola toka zaključno sa graničnim stanjima sistema.

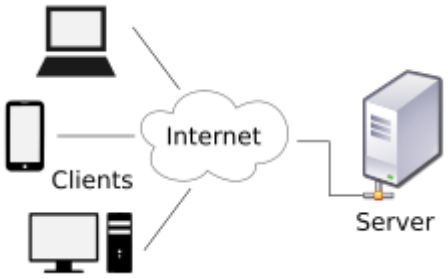
2. Arhitektura postojećeg sistema

2.1. Dvoslojna arhitektura

Dvoslojna arhitektura je vrsta slojevite arhitekture koja se sastoji od dva osnovna sloja: klijentskog sloja i serverskog sloja. Klijentski sloj predstavlja korisnički interfejs ili korisničko okruženje sa kojim korisnici vrše interakcije. To može biti desktop aplikacija, mobilna aplikacija ili browser. Klijentski sloj je odgovoran za prikazivanje podataka korisniku i prikupljanje korisničkih unosa. Serverski sloj je zadužen za obradu zahtjeva korisnika i pružanje odgovora. Serverski sloj obično sadrži aplikativnu logiku i pristup podacima. On obrađuje zahtjeve koji dolaze iz klijentskog sloja, izvršava potrebne operacije, pristupa bazi podataka i generiše odgovor koji se vraća klijentu. Najčešće je korišten HTTP protokol. Bitno je navesti da se dvoslojna arhitektura sastoji od 3 komponente koje su distribuirane u klijentski i serverski sloj:

- Korisnički interfejs
- Upravljanje procesima
- Upravljanje podacima

Prednost dvoslojne arhitekture je jednostavnost i lako razumijevanje, jer se sistem dijeli na samo dva osnovna sloja. Ova arhitektura može biti ograničena u pogledu skalabilnosti sistema i implementaciji složenih funkcionalnosti.



2.2. Troslojna arhitektura

Troslojna arhitektura organizuje sistem u tri odgovarajuća sloja. Napominjemo da su slojevi na različite načine imenovani u zavisnosti od literature ali njihovo objašnjenje i kontekst su identični. Ti slojevi su:

- Prezentacioni sloj (eng. Presentation Layer) - predstavlja korisnički interfejs sistema i omogućava korisnicima da komuniciraju sa

aplikacijom. To može biti browser, mobilna aplikacija, desktop aplikacija itd. Fokusira se na prikazivanje podataka na korisničkoj formi, prikupljanje korisničkih unosa, kao i prosljeđivanje zahtjeva poslovnom sloju.

- Poslovni sloj (eng. Business Layer) - sadrži poslovnu (aplikativnu) logiku aplikacije. On je odgovoran za obradu zahtjeva iz prezentacionog sloja, izvršavanje odgovarajućih operacija i manipulaciju podacima. Poslovni sloj implementira pravila poslovanja i logiku koja omogućava sistemsko ponašanje. Na ovom sloju se takođe nalaze procesi za validaciju i verifikaciju podataka.
- Sloj za pristup podacima (eng. Data Access Layer) - odgovoran za komunikaciju sa bazom podataka. Obezbjeđuje funkcionalnost za izvršavanje upita nad podacima, čitanje i pisanje u bazu podataka i obradu podataka. Bitna karakteristika je izolovanje i sakrivanje implementacije baze podataka od ostatka sistem.

Na slici su slojevi prikazani s lijeva na desno:

Troslojna arhitektura

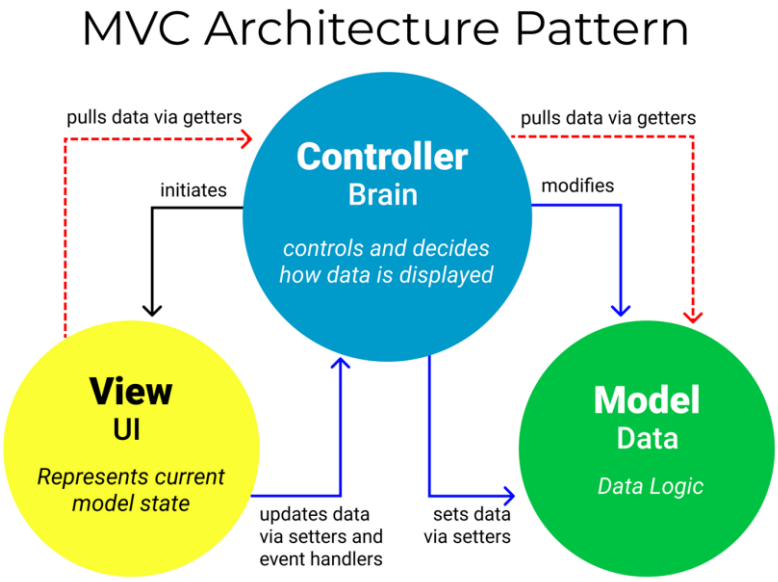
Troslojna arhitektura omogućava jasno odvajanje odgovornosti između slojeva, što olakšava održavanje, testiranje i skaliranje sistema. Svaki sloj se može razvijati i modifikovati nezavisno od drugih slojeva. Takođe, omogućava ponovno korišćenje komponenti i unapređenje sistema bez uticaja na druge slojeve. Ovakav obrazac je često prisutan u razvoju web aplikacija upravo zbog svojih dobrih osobina.

MVC (Model-View-Controller) je obrazac dizajna softvera koji se često koristi u razvoju aplikacija sa grafičkim korisničkim interfejsom. MVC razdvaja aplikaciju na tri osnovne komponente:

- **View** predstavlja korisnički interfejs, odnosno ono što korisnik vidi i sa čim interaguje. Ne smije da sadrži poslovnu logiku i pasivan je. View

može biti bilo koja forma prikaza, kao što su grafički interfejs ili web stranica.

- **Controller** je posrednik između modela i view-a. On prima korisničke inpute, obrađuje ih, upravlja tokom aplikacije i koordinira model i view.
- **Model** predstavlja sloj podataka i poslovne logike aplikacije. Ovdje se čuvaju podaci i definišu pravila za njihovu obradu. Uloga modela je čuvanje stanja aplikacije, kao i metode za pristup, izmjenu i pohranu podataka.



Šematski prikaz MVC modela

Moguće su i višeslojne arhitekture u skladu sa potrebama sistema. Kreiraju se na način da se na troslojnu arhitekturu dodaju novi slojevi (npr. Troslojna arhitektura sa dodatnim servisnim slojem).

3. Predložena arhitektura

3.1. Kratak pregled arhitekture I funkcionalnosti podsistema

Predloženo je da web aplikacija koristi troslojnu klijent-server arhitekturu koja razdvaja aplikaciju u tri sloja (sloj prezentacije, sloj logike aplikacije i sloj podataka). Za svaki od pomenutih slojeva planirana je implementacija pomoću sljedećih tehnologija:

1. **Sloj prezentacije**: frontend će biti realizovan pomoću Angular Framework, pomoću kojeg će se kreirati cjelokupan korisnički interfejs kako bi se korisniku omogućila lagana i intuitivna upotreba aplikacije. Korisnički

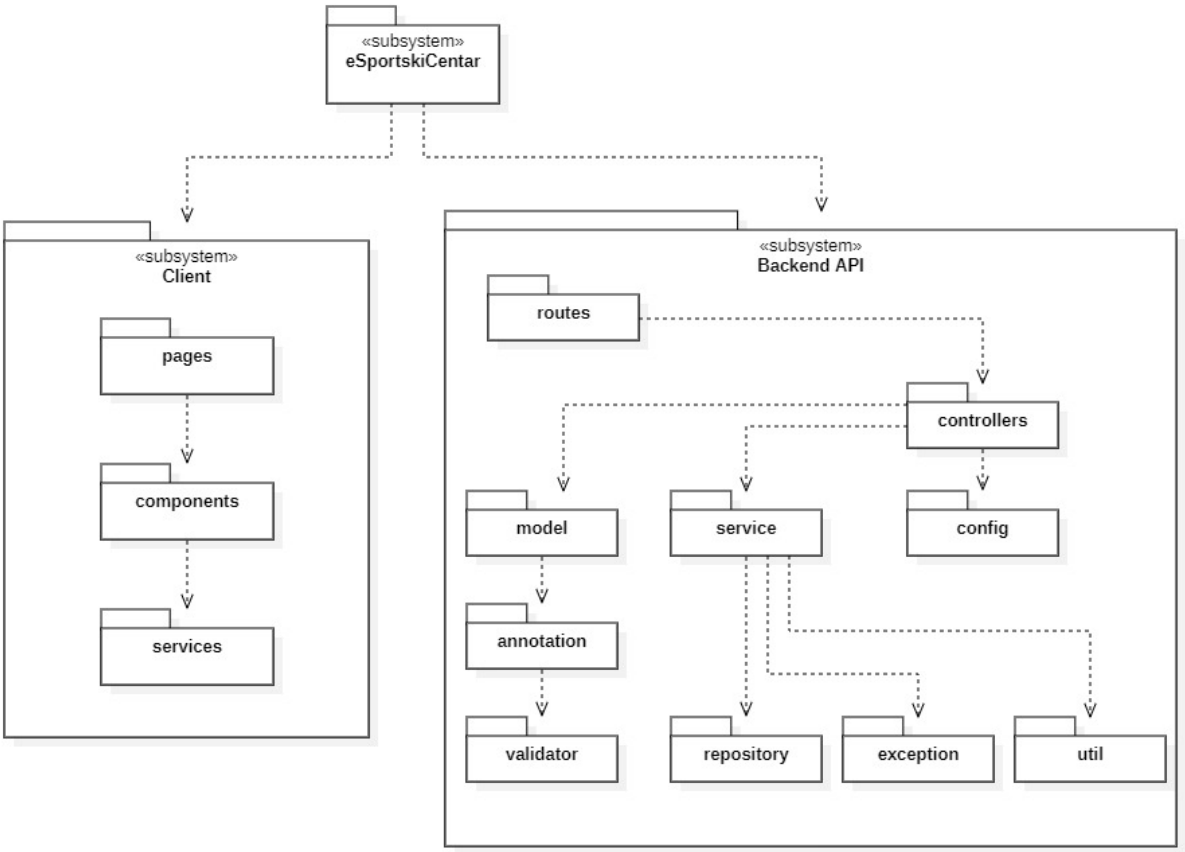
grafički interfejs (GUI) će prikazivati podatke modela korisniku u odgovarajućoj formi, te će vršiti pretvaranje ulaznih podataka i aktivnosti korisnika u odgovarajuće komande koje pozivaju funkcionalnosti aplikativne logike smještene na serveru.

2. **Sloj aplikacije**: backend će biti realizovan pomoću Java programskog jezika (Spring Boot). Ovaj sloj aplikacije će služiti kao posrednik između sloja prezentacije i sloja podataka i obavljaće svu logiku aplikacije: on će prihvatati korisničke inpute i transformisati ih u komande prema bazi podataka, manipulisaće podacima u bazi podataka i prikazivaće rezultate korisniku. Ovaj sloj je zadužen kako za provjeru, tako i za obradu unesenih podataka.

3. **Sloj podataka**: relaciona baza podataka, MySQL, smještena na DB serveru, koja odgovara na zahtjeve pristigle sa aplikativnog sloja. Sam odgovor zavisi od nivoa autorizacije, te mnogih integritetskih ograničenja.

Na klijentskoj strani će biti omogućen prikaz odgovarajuće forme za korisnika, te će korisničke aktivnosti biti interpretirane na odgovarajući način i izvršene pozivanjem funkcionalnosti smještenih na serverskoj strani. Serverska strana komunicira sa DBMS, vraćajući željeni rezultat na klijentsku stranu, gdje će on, na reprezentativan i efikasan način, biti prikazan korisniku.

3.2. Dekompozicija sistema



<< Dijagram dekompozicije sistema >>

Kao što se može vidjeti na slici, **Backend API** podsistem se sastoji od 3 podsistema:

1. **Routes** – predstavljaju rute za dohvatanje podataka iz perzistentnog sloja i na raspolaganju su drugom podsistemu Client.
2. **Controllers** – predstavljaju REST API kontrolere koji realizuju poslovnu logiku. Imaju pristup paketu *models* sa modelima objekata iz relacione baze podataka, paketu *config* koji služi za rad sa slikama kao i paketu *services* sa servisima koji sadrže operacije za rad sa bazom podataka. Servisi uz pomoć definisanih metoda pristupaju repozitorijumima iz paketa *repository* namijenjenim za objekte baze podataka te preko istih vrše manipulaciju nad njima. Servisi izuzetke bacaju iz *exceptions* paketa. Paketi *annotations* (pristupa mu paket sa modelima) i *util* (pristupaju mu servisi) sadrže manji broj metoda namijenjenih za upoređivanje. Kada je u pitanju validacija, u paketu *validators* nalaze se validatori za vrijeme, datum itd. Paket sa anotacijama zbog toga ima pristup ovom paketu u cilju validacije vrijednosti.

3. Skup paketa:
    - Model
    - Service
    - Config
    - Annotation
    - Validator
    - Repository
    - Exception
    - Util

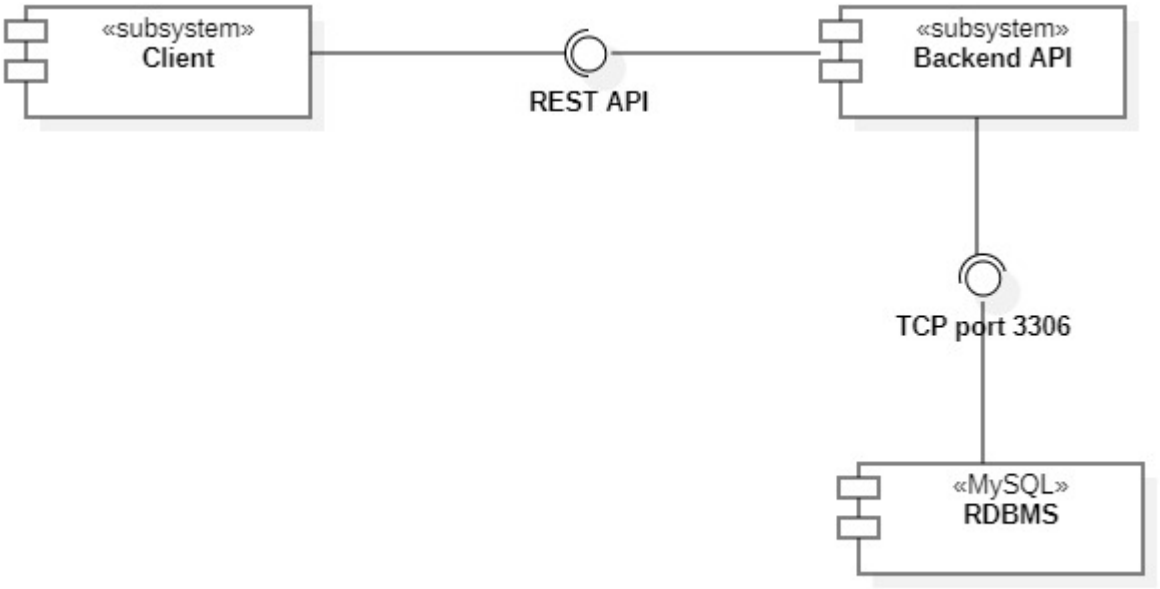
Client podsistem se sastoji od 3 paketa:

1. **Pages** – Angular stranice koje predstavljaju poglede, tj. ono što korisnik vidi preko web pregledača (browser-a). Funkcionišu na osnovu Angular komponenti kreirenih u frontend-u.
2. **Components** – Angular komponente.
3. **Services** – servisi sa osnovnim HTTP operacijama za obezbjeđivanje podataka Angular komponentama koje ih koriste.

3.3. HW/SW mapiranje

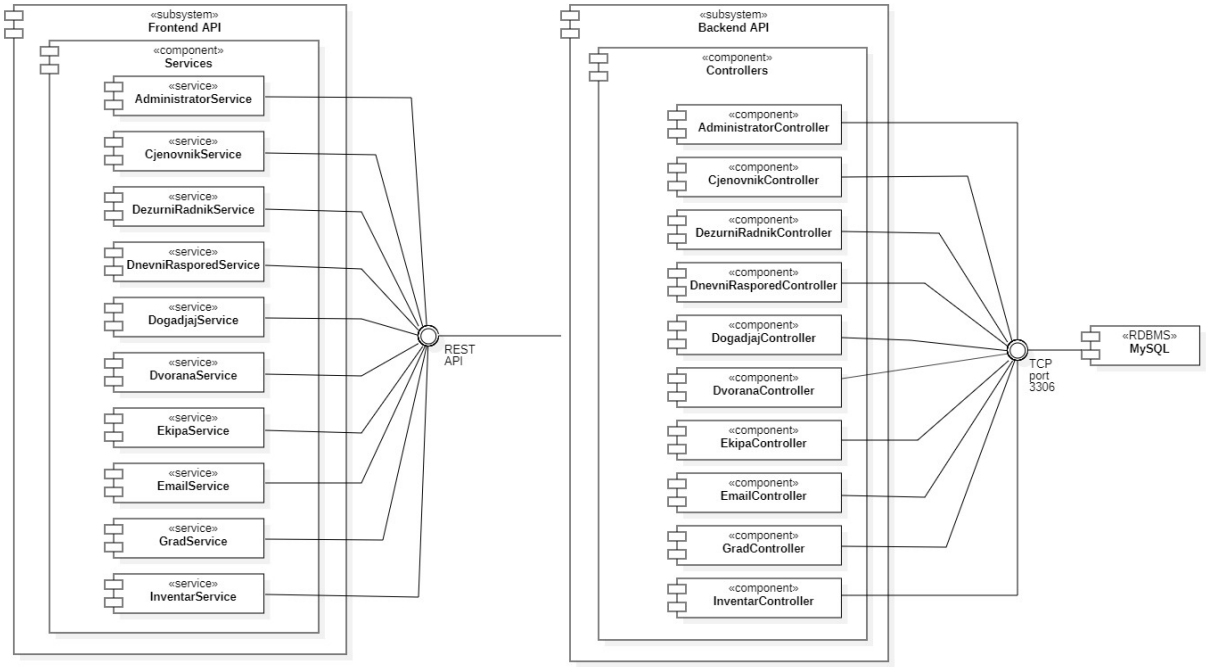
3.3.1. Dijagram komponenata

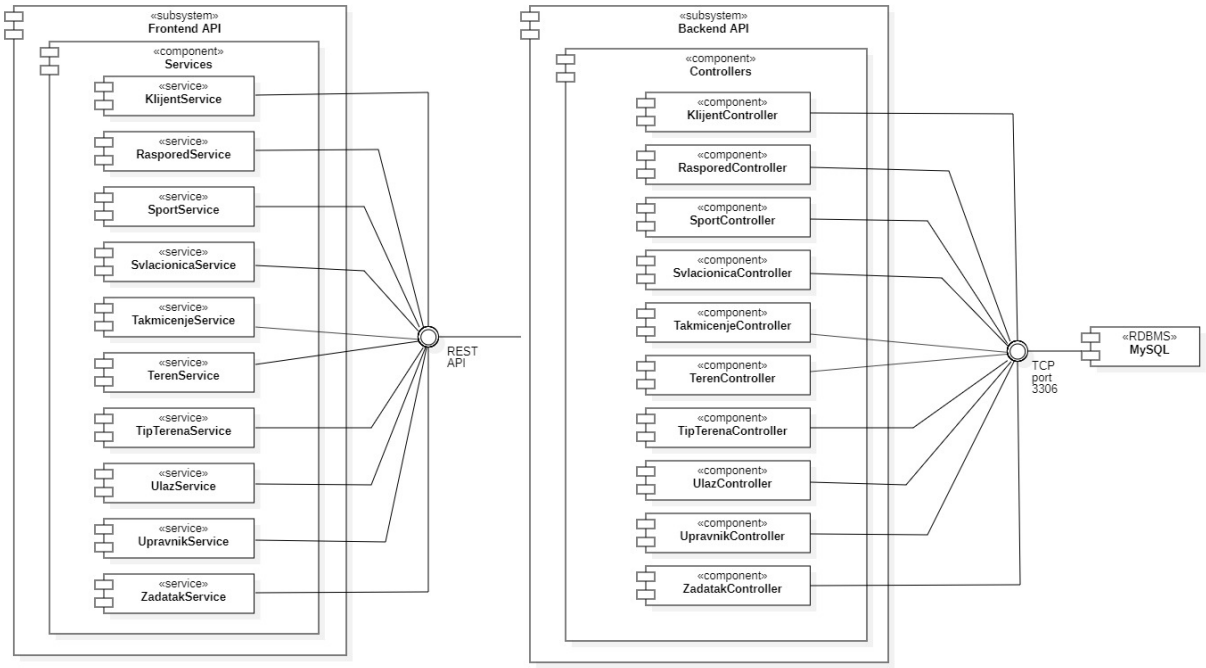
Na sljedećoj slici prikazan je dijagram komponenti sistema u UML notaciji:



Sistem se sastoji od 3 podsistema – Klijent, Backend API i RDBMS.

Backend API – služi da obezbijedi REST API za pristup i rad sa domenskim objektima sistema. Do svakog domenskog objekta u sistemu se može doći preko odgovarajuće putanje ili putem interfejsa na dijagramu komponenti koji sadrže metode analogne REST metodama Backend API-ja.





<< Dijagram komponenti za podsisteme (1. i 2. Dio) >>

REST API Backend API podsistema se sastoji od više kontrolera, a to su:

1. **AdministratorController** – obezbjeđuje operacije za rad sa administratorima.
2. **CjenovnikController** – obezbjeđuje operacije za rad sa cjenovnikom.
3. **DezurniRadnikController** – obezbjeđuje operacije za rad sa dežurnim radnicima.
4. **DnevniRasporedController** - obezbjeđuje operacije za rad sa dnevnim rasporedom.
5. **DogadjajController** - obezbjeđuje operacije za rad sa događajima.
6. **DvoranaController** - obezbjeđuje operacije za rad sa dvoranama.
7. **EkipaController** - obezbjeđuje operacije za rad sa ekipama.
8. **EmailContoller** - obezbjeđuje operacije za rad sa email-ovima.
9. **GradController** - obezbjeđuje operacije za rad sa gradovima.
10. **InventarController** - obezbjeđuje operacije za rad sa inventarima.
11. **KlijentController** - obezbjeđuje operacije za rad sa klijentima.
12. **RasporedController** - obezbjeđuje operacije za rad sa rasporedima.
13. **SportController** - obezbjeđuje operacije za rad sa sportovima.

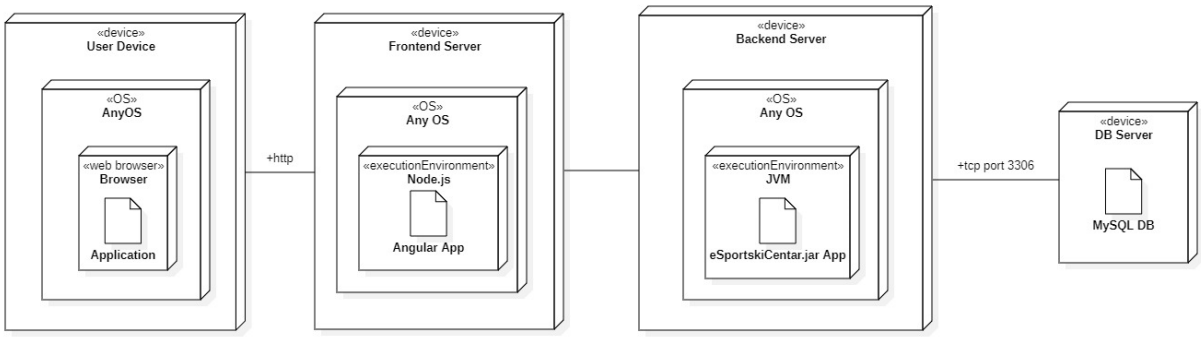
14. **SvlacionicaController** - obezbjeđuje operacije za rad sa svlačionicama.
15. **TakmicenjeController** - obezbjeđuje operacije za rad sa takmičenjima.
16. **TerenController** - obezbjeđuje operacije za rad sa terenima.
17. **TipTerenaController** - obezbjeđuje operacije za rad sa tipovima terena.
18. **UlazController** - obezbjeđuje operacije za rad sa ulazima.
19. **UpravnikController** - obezbjeđuje operacije za rad sa upravnicima.
20. **ZadatakController** - obezbjeđuje operacije za rad sa zadacima.

3.3.2. Dijagram razmještaja

Na dijagramu razmještaja je prikazan raspored artefakata sistema po fizičkim i logičkim čvorovima sistema. Na dijagramu se nalaze:

1. **User Device** – predstavlja krajnji uređaj kojim upravlja korisnik. Operativni sistem može biti proizvoljan, a korisniku treba da posjeduje internet konekciju i internet pretraživač za rad sa sistemom.
2. **Frontend Server** – služi za izvršavanje Client Angular aplikacije.
3. **Backend Server** – služi za dobavljanje podataka sa perzistentnog sloja putem Client podsistema.
4. **DB Server** – omogućava rad MySQL RDBMS podsistema.

Dakle, korisnici sistema koriste frontend aplikaciju koja se izvršava na frontend serveru. Frontend i backend server vrše međusobnu komunikaciju, dok backend server dobavlja podatke od DB servera.



<< Dijagram razmještaja >>

3.4. Perzistentni sloj

Naša aplikacija (eSportskiCentar) će koristiti relacionu bazu podataka. Preko konceptualnog modela relacione baze podataka biće prikazan perzistentni sloj, korištenjem IDEX0 i „crow's foot" notacije u MySQL Workbench alatu, omogućeno je potpuno strukturisanje podataka. Integritetska ograničenja biće implementirana u okviru aplikacije korišćenjem odgovarajuće programske logike.

Entitetski tip **KLIJENT** posjeduje sljedeće attribute:

- IdKlijent – jedinstveni identifikator klijenta
- Ime – ime klijenta
- Prezime – prezime klijenta
- BrojTelefona – broj telefona klijenta
- KorisnickoIme – korisničko ime klijenta pod kojim će klijent biti poznat
- Lozinka – lozinka naloga klijenta
- Email – email klijenta
- Blokiran – flag koji pokazuje da li je klijentov nalog blokiran

Entitetski tip **ADMINISTRATOR** posjeduje sljedeće atribute:

- IdAdministrator – jedinstveni identifikator administratora
- Ostali atributi entitetskog tipa KLIJENT, izuzev atributa "Blokiran"

Entitetski tip **UPRAVNIK** posjeduje sljedeće atribute:

- IdUpravnik – jedinstveni identifikator upravnika
- IdDvorana – jedinstveni identifikator dvorane za koju je zadužen
- Ostali atributi entitetskog tipa KLIJENT

Entitetski tip **DEŽURNI RADNIK** posjeduje sljedeće atribute:

- IdDezurniRadnik – jedinstveni identifikator dežurnog radnika

- Ostali atributi entitetskog tipa KLIJENT

Entitetski tip **SPORT** posjeduje sljedeće atribute:

- IdSport – jedinstveni identifikator sporta
- NazivSporta – naziv sporta
- Duzina – dužina terena na kom se sport igra
- Sirina – širina terena na kom se sport igra
- IdTipTerena – jedinstveni identifikator tipa terena

Entitetski tip **TAKMIČENJE** posjeduje sljedeće atribute:

- IdTakmicenje – jedinstveni identifikator takmičenja
- Vrsta takmičenja – tip takmičenja (liga/turnir)

Entitetski tip **EKIPA** posjeduje sljedeće atribute:

- IdEkipe – jedinstveni identifikator ekipe
- NazivEkipe – naziv ekipe
- IdSport – identifikator sporta u kojem se takmiči ekipa

Entitetski tip **DOGAĐAJ** posjeduje sljedeće atribute:

- IdDogadjaj – jedinstveni identifikator događaja
- IdTakmicenje – jedinstveni identifikator takmičenja koje se odigrava
- IdDomaceEkipe – jedinstveni identifikator domaće ekipe
- IdGostujuceEkipe – jedinstveni identifikator gostujuće ekipe
- VrijemeOd – vrijeme početka događaja
- VrijemeDo – vrijeme završetka događaja
- InfoDogadjaja – dodatne informacije o događaju
- IdDnevniRaspored – jedinstveni identifikator dnevnog rasporeda
- IdTeren – jedinstveni identifikator terena na kom će biti održan događaj

- Cijena – cijena za dati događaj
- IdKlijent – jedinstveni identifikator klijenta
- IdSport – jedinstveni identifikator sporta
- Odobren – flag koji pokazuje da li je održavanje događaja odobreno

Entitetski tip **ZADATAK** posjeduje sljedeće atribute:

- IdZadatak – jedinstveni identifikator zadatka
- Naslov – naslov zadatka
- DatumKreiranja – datum kreiranja zadatka
- RokIzvrsenja – krajnji rok za izvršenje zadatka
- Info – dodatne informacije o zadatku
- IdUpravnik – jedinstveni identifikator upravnika koji dodjeljuje zadatak
- IdDezurniRadnik – jedinstveni identifikator dežurnog radnika koji treba da obavi zadatak
- Zavrsen – flag koji pokazuje da li je zadatak završen

Entitetski tip **GRAD** posjeduje sljedeće atribute:

- IdGrad – jedinstveni identifikator grada
- NazivGrada – naziv grada

Entitetski tip **DVORANA** posjeduje sljedeće atribute:

- IdDvorana – jedinstveni identifikator dvorane
- NazivDvorane – naziv dvorane
- IdGrad – grad u kome se nalazi dvorana
- Kapacitet – kapacitet dvorane (broj sjedišta)
- Info – dodatne informacije o dvorani

Entitetski tip **INVENTAR** posjeduje sljedeće atribute:

- IdInventar – jedinstveni identifikator inventara
- IdDvorana – jedinstveni identifikator dvorane na koju se odnosi
- Naziv – naziv inventara
- Opis – dodatni opis inventara

Entitetski tip **ULAZ** posjeduje sljedeće atribute:

- IdUlaz – jedinstveni identifikator ulaza u dvoranu
- IdDvorana – jedinstveni identifikator odgovarajuće dvorane
- NazivUlaza – naziv ulaza
- BrojUlaza – broj određenog ulaza u dvoranu
- Dostupan – flag koji pokazuje da li je određeni ulaz u dvoranu dostupan

Entitetski tip **SVLAČIONICA** posjeduje sljedeće atribute:

- IdSvlacionica – jedinstveni identifikator svlačionice
- IdDvorana – lokacija svlačionice
- BrojSvlacionice – broj svlačionice
- Dostupna – flag koji pokazuje da li je dvorana dostupna u određenom trenutku

Entitetski tip **RASPORED** posjeduje sljedeće atribute:

- IdRaspored – jedinstveni identifikator rasporeda događaja
- TipRasporeda – tip rasporeda

Entitetski tip **DNEVNI RASPORED** posjeduje sljedeće atribute:

- IdDnevniRaspored – jedinstveni identifikator dnevnog rasporeda
- Datum – datum za raspored
- IdRaspored – jedinstveni identifikator odgovarajućeg rasporeda

Entitetski tip **TEREN** posjeduje sljedeće atribute:

- IdTeren – jedinstveni identifikator terena
- IdTipTerena – jedinstveni identifikator tipa terena
- NazivTerena – naziv terena na kom će se odvijati neki događaj
- Info – dodatne informacije o terenu
- IdDvorana – jedinstveni identifikator dvorane
- Slika – fotografija terena
- Duzina – dužina terena
- Sirina – širina terena
- Dostupan – flag koji pokazuje da li je teren dostupan

Entitetski tip **TIP TERENA** posjeduje sljedeće atribute:

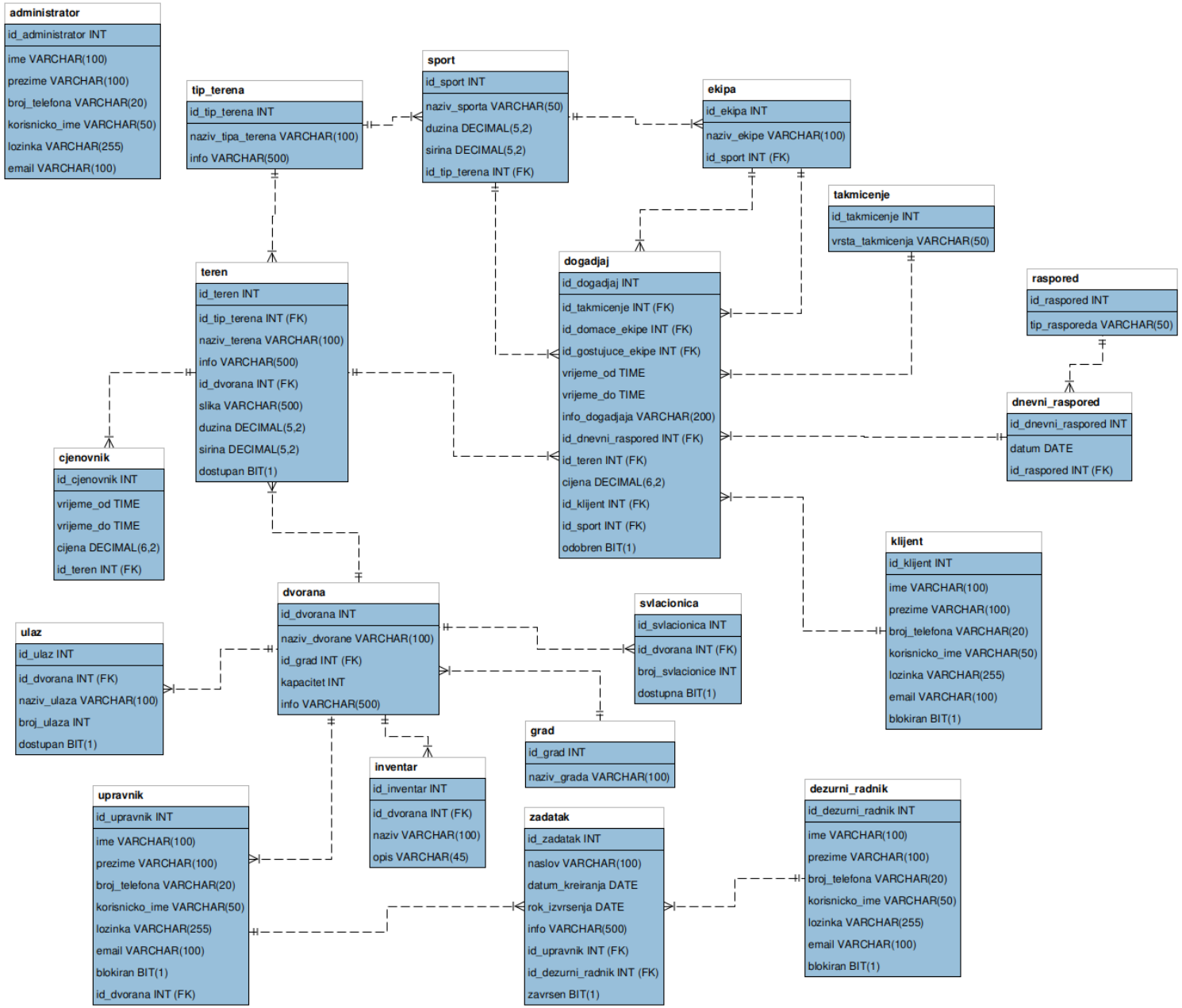
- IdTipTerena – jedinstveni identifikator tipa terena
- NazivTipaTerena – naziv tipa terena
- Info – dodatne informacije o tipu

Entitetski tip **CJENOVNIK** posjeduje sljedeće atribute:

- IdCjenovnik – jedinstveni identifikator cjenovnika
- VrijemeOd – vrijeme početka termina
- VrijemeDo – vrijeme kraja termina
- Cijena – cijena za dati termin
- IdTeren – jedinstveni identifikator odabranog terena za događaj

eSportskiCentar Verzija 2.0

Dizajn sistema



3.5. Kontrola prava pristupa i sigurnost

3.5.1. Podsistem za administratora

Klasa	*Operacija*	*Uloga*
		Administrator
Administrator	getAdministrators( )	✓
	addAdministrator( )	✓
Upravnik	getUpravniks( )	✓
	addUpravnik( )	✓
	updateUpravnik( )	✓
	deleteUpravnik( )	✓
Dežurni radnik	getDezurniRadniks( )	✓
	addDezurniRadnik( )	✓
	updateDezurniRadnik( )	✓
	deleteDezurniRadnik( )	✓
Dvorana	getDvoranas( )	✓
	getDvoranaById( )	✓
	addDvorana( )	✓
	updateDvorana( )	✓
	deleteDvorana( )	✓
Događaj	getDogadjas( )	✓
	getDogadjajsOfUser( )	✓
	getDogadjajById( )	✓
	addDogadjaj( )	✓
	updateDogadjaj( )	✓
	deleteDogadjaj( )	✓
Raspored	getRasporeds( )	✓
	getRasporedsById( )	✓
	addRaspored( )	✓
	updateRaspored( )	✓
	deleteRaspored( )	✓
Dnevni raspored	getDnevniRasporeds( )	✓
	getDnevniRasporedById()	✓
	addDnevniRaspored( )	✓
	updateDnevniRaspored()	✓
	deleteDnevniRaspored( )	✓
Takmičenje	getTakmicenjes( )	✓
	getTakmicenjeById( )	✓
	addTakmicenje( )	✓
	updateTakmicenje( )	✓
	deleteTakmicenje( )	✓

Zadatak	getZadataks( )	
	getZadatakById( )	
	addZadatak( )	
	updateZadatak( )	
	deleteZadatak( )	
Svlačionica	getSvlacionicas( )	✓
	getSvlacionicaById( )	✓
	addSvlacionica( )	✓
	updateSvlacionica( )	✓
	deleteSvlacionica( )	✓
Sport	getSports( )	✓
	getSportById( )	✓
	addSport( )	✓
	updateSport( )	✓
	deleteSport( )	✓
Ekipa	getEkipas( )	✓
	getEkipaById( )	✓
	addEkipa( )	✓
	updateEkipa( )	✓
	deleteEkipa( )	✓
Grad	getGrads( )	✓
	getGradById( )	✓
	addGrad( )	✓
	updateGrad( )	✓
	deleteGrad( )	✓
Teren	getTerens( )	✓
	getTerenById( )	✓
	addTeren( )	✓
	updateTeren( )	✓
	deleteTeren( )	✓
Tip terena	getTipTerenas( )	✓
	getTipTerenaById( )	✓
	addTipTerena( )	✓
	updateTipTerena( )	✓
	deleteTipTerena( )	✓
Ulaz	getUlazs( )	✓
	getUlazById( )	✓
	addUlaz( )	✓
	updateUlaz( )	✓
	deleteUlaz( )	✓

Inventar	getInventars( )	
	getInventarById( )	
	addInventar( )	
	updateInventar( )	
	deleteInventar( )	
Cjenovnik	getCjenovniks( )	
	getCjenovnikById( )	
	addCjenovnik( )	
	updateCjenovnik( )	
	deleteCjenovnik( )	

3.5.2. Podsistem za upravnika

Klasa	*Operacija*	*Uloga*
		Upravnik
Administrator	getAdministrators( )	
	addAdministrator( )	
Upravnik	getUpravniks( )	
	addUpravnik( )	
	updateUpravnik( )	
	deleteUpravnik( )	
Dežurni radnik	getDezurniRadniks( )	
	addDezurniRadnik( )	
	updateDezurniRadnik( )	
	deleteDezurniRadnik( )	
Dvorana	getDvoranas( )	
	getDvoranaById( )	
	addDvorana( )	
	updateDvorana( )	
	deleteDvorana( )	
Događaj	getDogadjas( )	✓
	getDogadjajsOfUser( )	✓
	getDogadjajById( )	✓
	addDogadjaj( )	✓
	updateDogadjaj( )	✓
	deleteDogadjaj( )	✓
Raspored	getRasporeds( )	✓
	getRasporedsById( )	✓
	addRaspored( )	✓
	updateRaspored( )	✓
	deleteRaspored( )	✓
Dnevni raspored	getDnevniRasporeds( )	✓
	getDnevniRasporedById()	✓
	addDnevniRaspored( )	✓
	updateDnevniRaspored()	✓
	deleteDnevniRaspored( )	✓
Takmičenje	getTakmicenjes( )	
	getTakmicenjeById( )	
	addTakmicenje( )	
	updateTakmicenje( )	
	deleteTakmicenje( )	

Zadatak	getZadataks( )	✓
	getZadatakById( )	✓
	addZadatak( )	✓
	updateZadatak( )	✓
	deleteZadatak( )	✓
Svlačionica	getSvlacionicas( )	✓
	getSvlacionicaById( )	✓
	addSvlacionica( )	✓
	updateSvlacionica( )	✓
	deleteSvlacionica( )	✓
Sport	getSports( )	
	getSportById( )	
	addSport( )	
	updateSport( )	
	deleteSport( )	
Ekipa	getEkipas( )	
	getEkipaById( )	
	addEkipa( )	
	updateEkipa( )	
	deleteEkipa( )	
Grad	getGrads( )	
	getGradById( )	
	addGrad( )	
	updateGrad( )	
	deleteGrad( )	
Teren	getTerens( )	✓
	getTerenById( )	✓
	addTeren( )	✓
	updateTeren( )	✓
	deleteTeren( )	✓
Tip terena	getTipTerenas( )	
	getTipTerenaById( )	
	addTipTerena( )	
	updateTipTerena( )	
	deleteTipTerena( )	
Ulaz	getUlazs( )	✓
	getUlazById( )	✓
	addUlaz( )	✓
	updateUlaz( )	✓
	deleteUlaz( )	✓

Inventar	getInventars( )	✓
	getInventarById( )	✓
	addInventar( )	✓
	updateInventar( )	✓
	deleteInventar( )	✓
Cjenovnik	getCjenovniks( )	✓
	getCjenovnikById( )	✓
	addCjenovnik( )	✓
	updateCjenovnik( )	✓
	deleteCjevnovnik( )	✓

3.5.3. Podsistem za dežurnog radnika

Klasa	*Operacija*	*Uloga*
		Dežurni radnik
Administrator	getAdministrators( )	
	addAdministrator( )	
Upravnik	getUpravniks( )	
	addUpravnik( )	
	updateUpravnik( )	
	deleteUpravnik( )	
Dežurni radnik	getDezurniRadniks( )	
	addDezurniRadnik( )	
	updateDezurniRadnik( )	
	deleteDezurniRadnik( )	
Dvorana	getDvoranas( )	
	getDvoranaById( )	
	addDvorana( )	
	updateDvorana( )	
	deleteDvorana( )	
Događaj	getDogadjas( )	
	getDogadjajsOfUser( )	
	getDogadjajById( )	
	addDogadjaj( )	
	updateDogadjaj( )	
	deleteDogadjaj( )	
Raspored	getRasporeds( )	
	getRasporedsById( )	
	addRaspored( )	
	updateRaspored( )	
	deleteRaspored( )	
Dnevni raspored	getDnevniRasporeds( )	
	getDnevniRasporedById()	
	addDnevniRaspored( )	
	updateDnevniRaspored()	
	deleteDnevniRaspored( )	
Takmičenje	getTakmicenjes( )	
	getTakmicenjeById( )	
	addTakmicenje( )	
	updateTakmicenje( )	
	deleteTakmicenje( )	

Zadatak	getZadataks( )	✓
	getZadatakById( )	✓
	addZadatak( )	
	updateZadatak( )	✓
	deleteZadatak( )	
Svlačionica	getSvlacionicas( )	
	getSvlacionicaById( )	
	addSvlacionica( )	
	updateSvlacionica( )	
	deleteSvlacionica( )	
Sport	getSports( )	
	getSportById( )	
	addSport( )	
	updateSport( )	
	deleteSport( )	
Ekipa	getEkipas( )	
	getEkipaById( )	
	addEkipa( )	
	updateEkipa( )	
	deleteEkipa( )	
Grad	getGrads( )	
	getGradById( )	
	addGrad( )	
	updateGrad( )	
	deleteGrad( )	
Teren	getTerens( )	
	getTerenById( )	
	addTeren( )	
	updateTeren( )	
	deleteTeren( )	
Tip terena	getTipTerenas( )	
	getTipTerenaById( )	
	addTipTerena( )	
	updateTipTerena( )	
	deleteTipTerena( )	
Ulaz	getUlazs( )	
	getUlazById( )	
	addUlaz( )	
	updateUlaz( )	
	deleteUlaz( )	

Inventar	getInventars( )	
	getInventarById( )	
	addInventar( )	
	updateInventar( )	
	deleteInventar( )	
Cjenovnik	getCjenovniks( )	
	getCjenovnikById( )	
	addCjenovnik( )	
	updateCjenovnik( )	
	deleteCjenovnik( )	

3.5.4. Podsistem za klijenta

Klasa	*Operacija*	*Uloga*
		Klijent
Administrator	getAdministrators( )	
	addAdministrator( )	
Upravnik	getUpravniks( )	
	addUpravnik( )	
	updateUpravnik( )	
	deleteUpravnik( )	
Dežurni radnik	getDezurniRadniks( )	
	addDezurniRadnik( )	
	updateDezurniRadnik( )	
	deleteDezurniRadnik( )	
Dvorana	getDvoranas( )	✓
	getDvoranaById( )	
	addDvorana( )	
	updateDvorana( )	
	deleteDvorana( )	
Događaj	getDogadjas( )	✓
	getDogadjajsOfUser( )	✓
	getDogadjajById( )	
	addDogadjaj( )	
	updateDogadjaj( )	
	deleteDogadjaj( )	
Raspored	getRasporeds( )	
	getRasporedsById( )	
	addRaspored( )	
	updateRaspored( )	
	deleteRaspored( )	
Dnevni raspored	getDnevniRasporeds( )	
	getDnevniRasporedById()	
	addDnevniRaspored( )	
	updateDnevniRaspored()	
	deleteDnevniRaspored( )	
Takmičenje	getTakmicenjes( )	
	getTakmicenjeById( )	
	addTakmicenje( )	
	updateTakmicenje( )	
	deleteTakmicenje( )	

Zadatak	getZadataks( )	
	getZadatakById( )	
	addZadatak( )	
	updateZadatak( )	
	deleteZadatak( )	
Svlačionica	getSvlacionicas( )	✓
	getSvlacionicaById( )	
	addSvlacionica( )	
	updateSvlacionica( )	
	deleteSvlacionica( )	
Sport	getSports( )	
	getSportById( )	
	addSport( )	
	updateSport( )	
	deleteSport( )	
Ekipa	getEkipas( )	
	getEkipaById( )	
	addEkipa( )	
	updateEkipa( )	
	deleteEkipa( )	
Grad	getGrads( )	
	getGradById( )	
	addGrad( )	
	updateGrad( )	
	deleteGrad( )	
Teren	getTerens( )	✓
	getTerenById( )	
	addTeren( )	
	updateTeren( )	
	deleteTeren( )	
Tip terena	getTipTerenas( )	
	getTipTerenaById( )	
	addTipTerena( )	
	updateTipTerena( )	
	deleteTipTerena( )	
Ulaz	getUlazs( )	✓
	getUlazById( )	
	addUlaz( )	
	updateUlaz( )	
	deleteUlaz( )	

Inventar	getInventars( )	
	getInventarById( )	
	addInventar( )	
	updateInventar( )	
	deleteInventar( )	
Cjenovnik	getCjenovniks( )	
	getCjenovnikById( )	
	addCjenovnik( )	
	updateCjenovnik( )	
	deleteCjenovnik( )	

3.6. Konkurentnost i kontrola toka

Da bi se postigao potreban nivo konkurentnosti i kontrole toka, sve akcije dohvatanja podataka iz baze moraju biti neblokirajuće. Korisnički grafički interfejs takođe mora biti neblokirajući. Na taj način se postiže da korisnik dobija odgovor odmah i bez čekanja i da može da radi više operacija istovremeno.

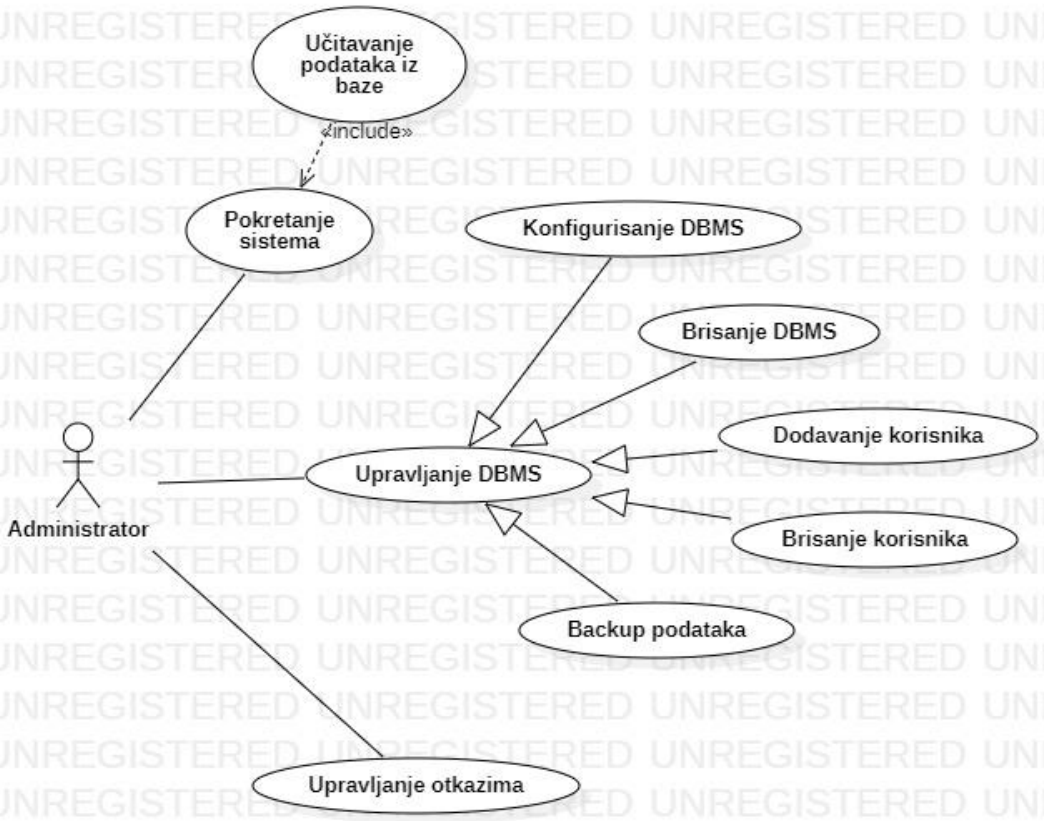
3.7. Granična stanja sistema

Granična stanja sistema modeluju se graničnim slučajevima upotrebe (boundary use cases). Alternativno se koristi i termin administrativni slučajevi upotrebe, jer je korespondentni učesnik najčešće administrator sistema. Granični slučajevi upotrebe tipično se identifikuju tokom dizajna, kroz analizu podsistema i perzistentnih objekata:

– **konfiguracija**: za svaki perzistentni objekat se analiziraju slučajevi upotrebe u kojima se perzistentni objekti kreiraju, uništavaju, arhiviraju, ... Ako ne postoji odgovarajući slučaj upotrebe u kojem se dešavaju granična stanja perzistentnih objekata, treba dodati novi granični slučaj upotrebe;

– **terminiranje/završetak rada**: za svaku komponentu treba dodati granične slučajeve upotrebe za start, shut down i konfiguraciju; nekad jedan slučaj upotrebe može da kontroliše istu aktivnost nad više komponenata;

– **manipulacija izuzecima**: za svaku komponentu treba analizirati moguće otkaze i načine na koje komponenta treba da se ponaša u slučaju izuzetaka; može da se modeluje dodatnim SU koji proširuje funkcionalnosti osnovnog SU. Dobro projektovan sistem ima sposobnost “predviđanja” otkaza i ima mehanizme za manipulaciju i oporavak od otkaza.



UNREGISTERED